



빅데이터 엔지니어링

2회차

아키텍트의 도구 상자

ICT폴리텍대학 장지연 교수

아키텍트의 도구 상자

도구를 나열하지 않는다. 아키텍트가 하는 네 가지 행위를 보고, 그 자리에 무엇이 들어가는지 본다.

1교시 선언 · 2교시 기록 ·
3교시 접속과 관찰

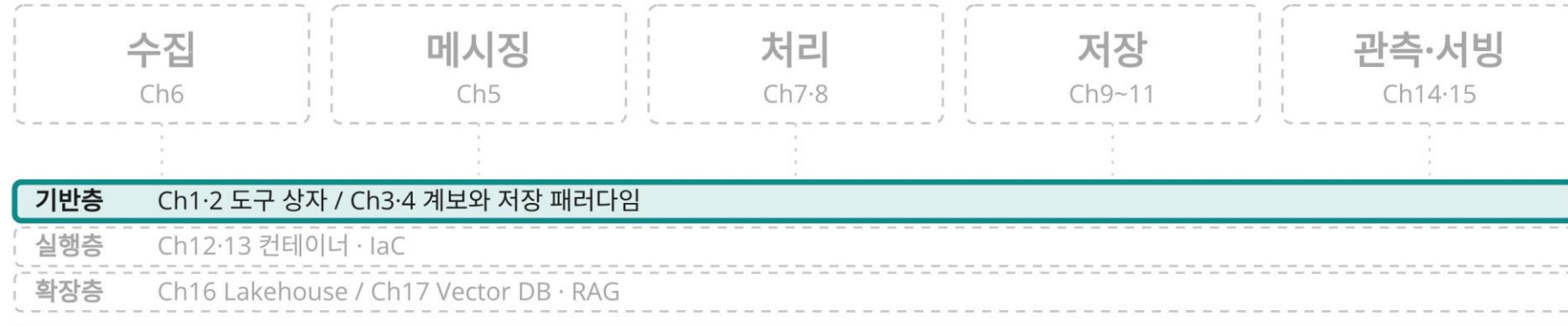
강의 2.5h + 실습 1.5h — 오늘
실습은 문서 체계와 서버 접속

빅데이터 엔지니어링 · 2주차



1교시 — 선언

유형 — 데이터가 지나가는 다섯 계층



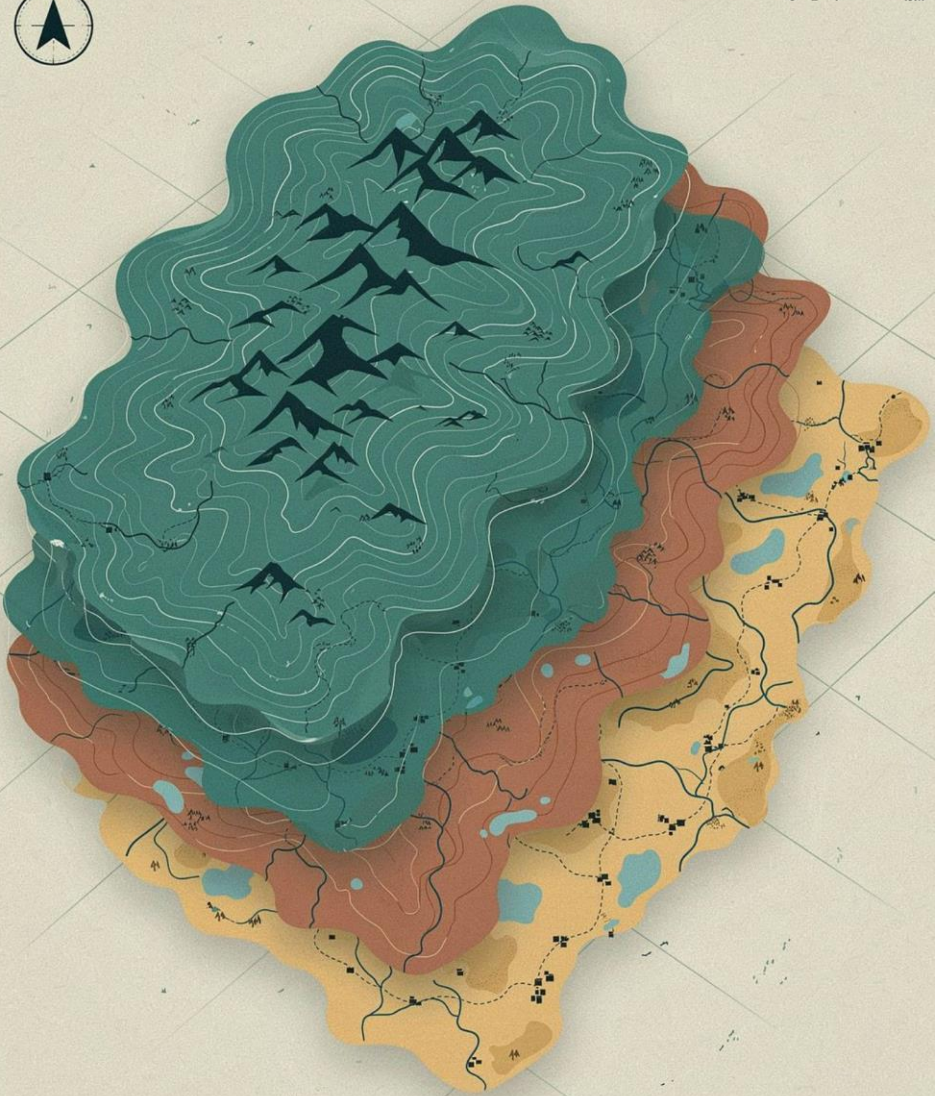
실물 — 실습 시스템 TFDS-Edu



LAYERED MAP



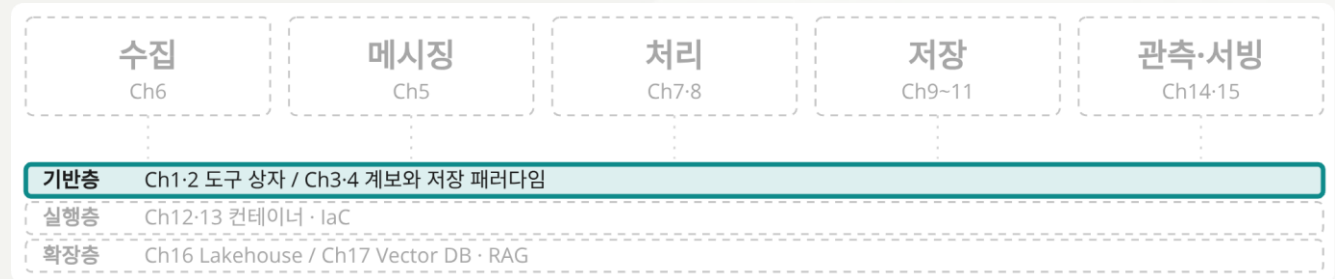
0 2 1 15m



BASE LAYER • RIVERS • SETTLEMENTS

오늘 우리는 이 지도의 어디에 있는가

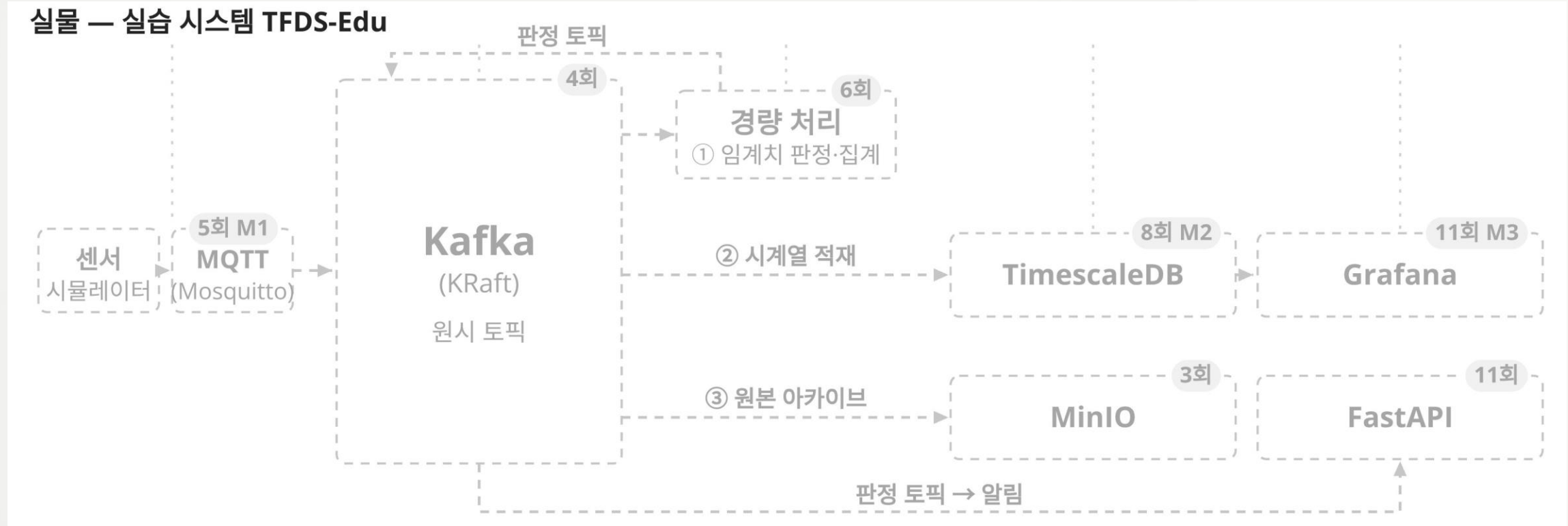
이번 회차는 다섯 계층 어디에도 칸이 없다. 그 아래 바탕에 있다.



- ☐ 상단 유형 지도 — 수집·메시징·처리·저장·관측/서빙. 강조는 계층이 아니라 기반층
- ☐ 12회차 중 바탕을 강조하는 유일한 회차
- ☐ 도구는 특정 계층의 것이 아니라 전 계층에 걸린다

오늘 채우는 칸은 없다

관통 실습의 칸은 3주차부터 하나씩 커진다. 오늘 세우는 것은 12주 내내 쓸 작업 방식이다.



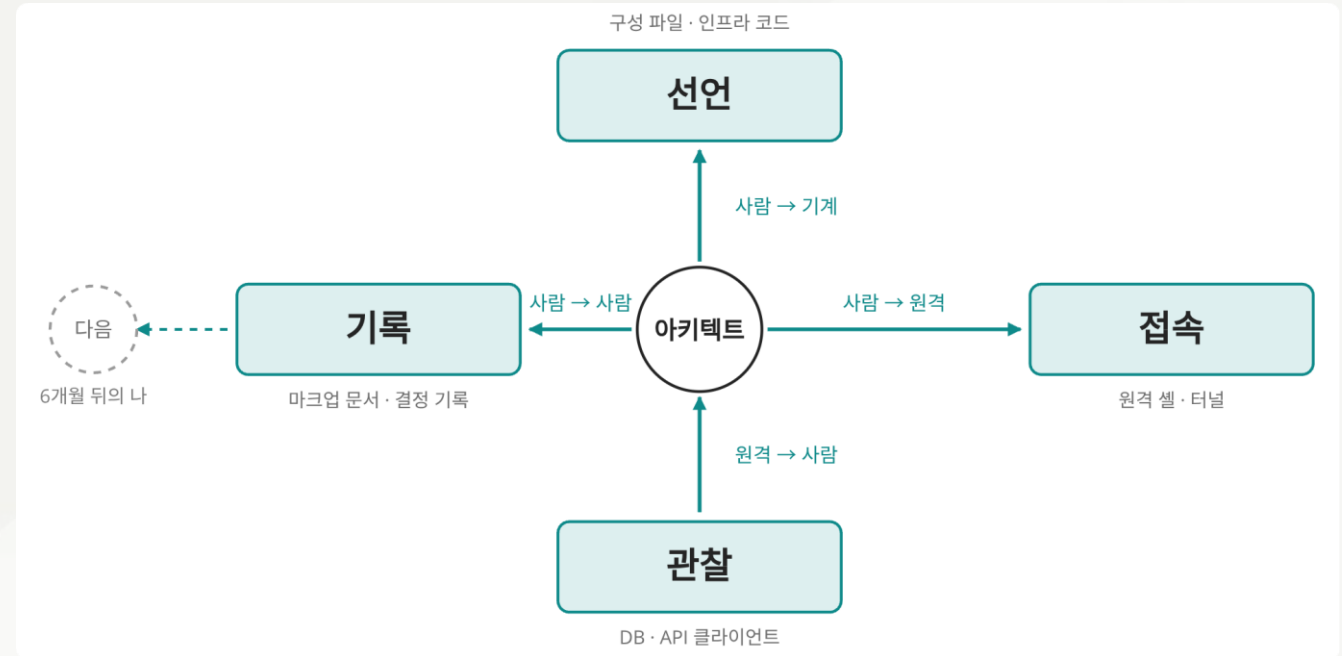
하단 TFDS-Edu 배선 — 커진 칸
0개, 1주차와 같다

1주차가 만든 것은 그릇, 2주차가
만드는 것은 그 안에 무엇을
어떻게 넣는가

칸을 채우지 않는 회차는 오늘이
마지막이다

도구가 아니라 네 가지 행위

아키텍트가 시스템과 주고받는 일은 네 가지로 묶인다.



선언

무엇을 원하는지
기계에게

기록

왜 그렇게 했는지
사람에게

접속

저쪽 시스템에
손을 대는 통로

관찰

안에 무엇이
들었는지 보는 창

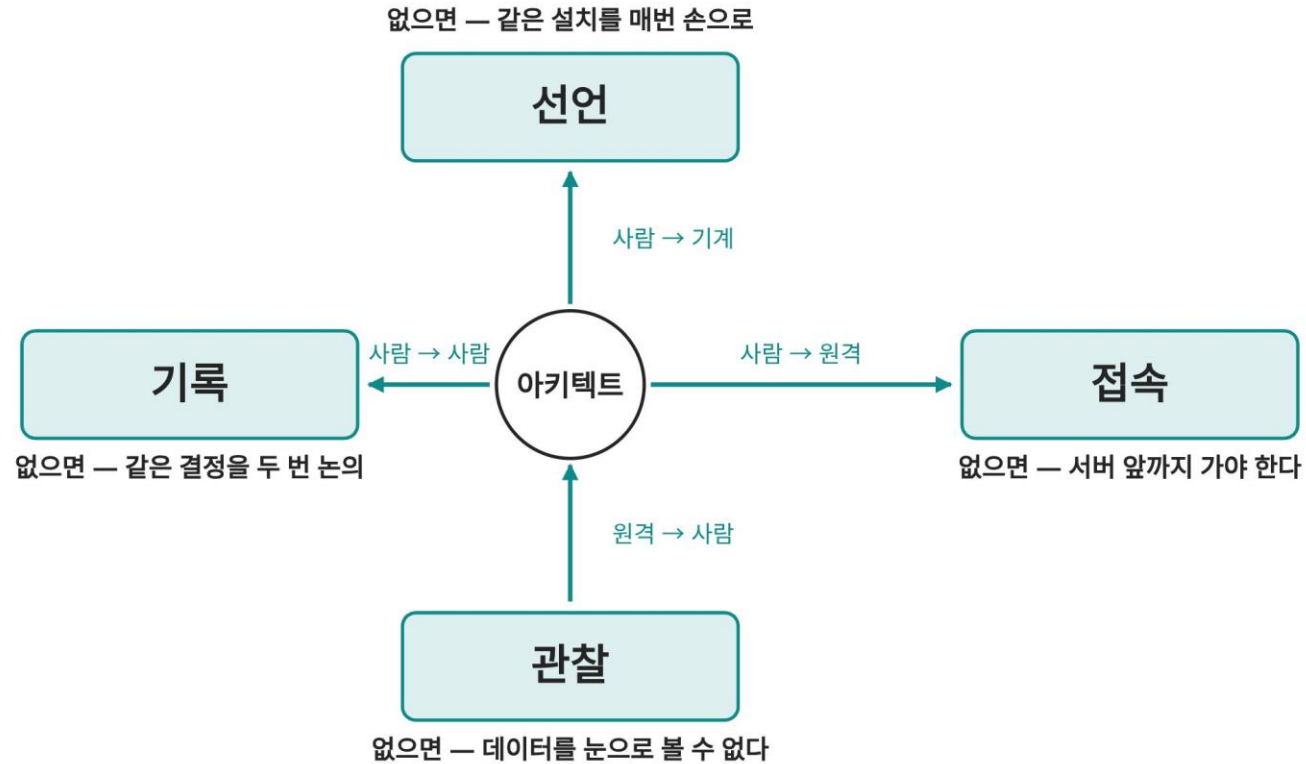
도구는 이 네 자리에 들어가는 사례다. 자리가 먼저다

도구가 없으면 무엇을 손으로 하게 되는가

각 행위의 도입 판단 기준은 하나다. 손으로 해도 되는 규모가 어디까지인가.

각 자리에 도구가 없으면 무엇을 손으로 하게 되는가

도입 판단 기준은 하나다 — 손으로 해도 되는 규모가 어디까지인가



손으로 해도 되는 규모가 있다 — 그 경계를 아는 것이 판단이다

선언이 없으면

같은 설치를 매번 손으로. 구성요소가 늘수록 재현이 깨진다

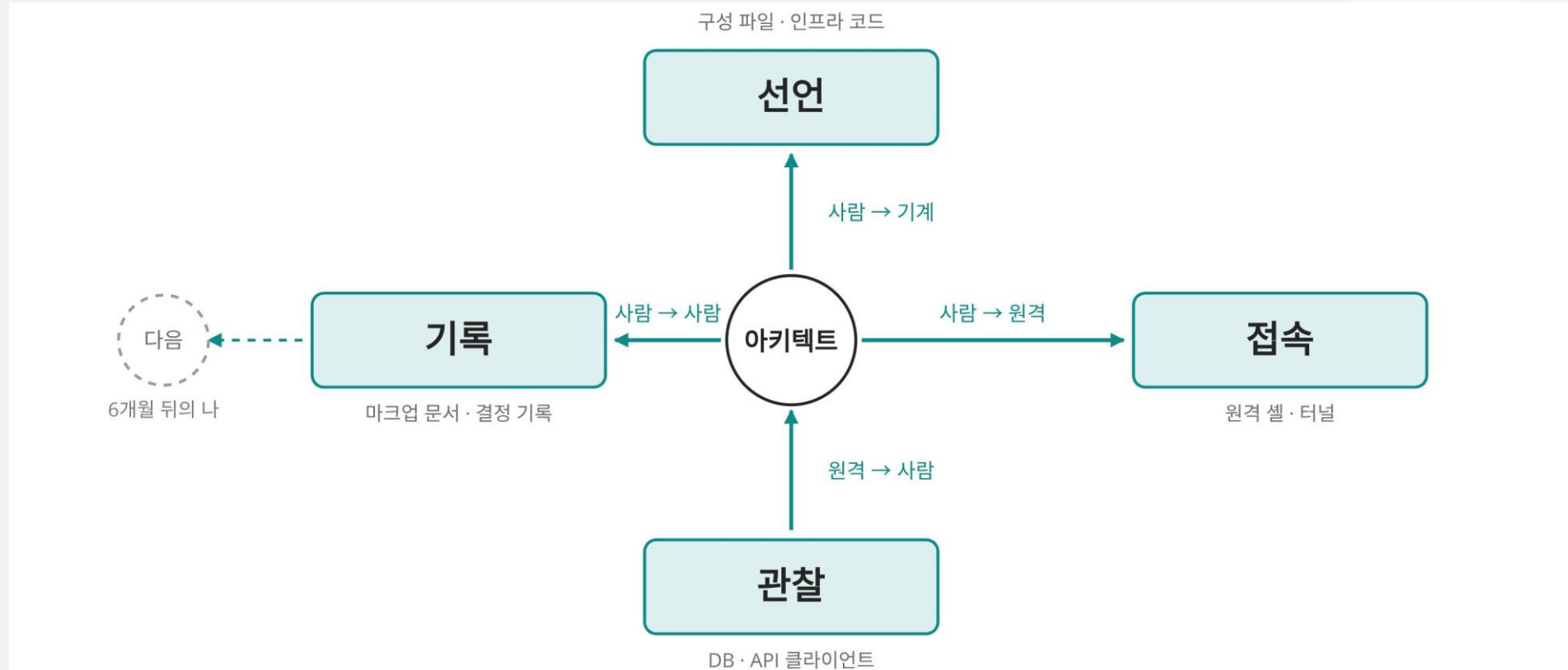
기록이 없으면

같은 결정을 두 번 논의. 참여자가 늘수록 비용이 는다

접속·관찰이 없으면

서버 앞에 가야 하고, 데이터를 눈으로 볼 수 없다

네 자리에 무엇이 들어가는가



선언

구성 파일 형식(YAML) · 컨테이너 구성 · 인프라 코드

기록

마크업 문서 · 결정 기록 · 텍스트 다이어그램

접속과 관찰

원격 셀 · 터널 · DB 클라이언트 · API 클라이언트

도구는 바뀌고 행위는 남는다

오늘 배우는 제품 이름 넷은 5년 뒤 달라질 수 있지만, 네 자리는 그대로다.

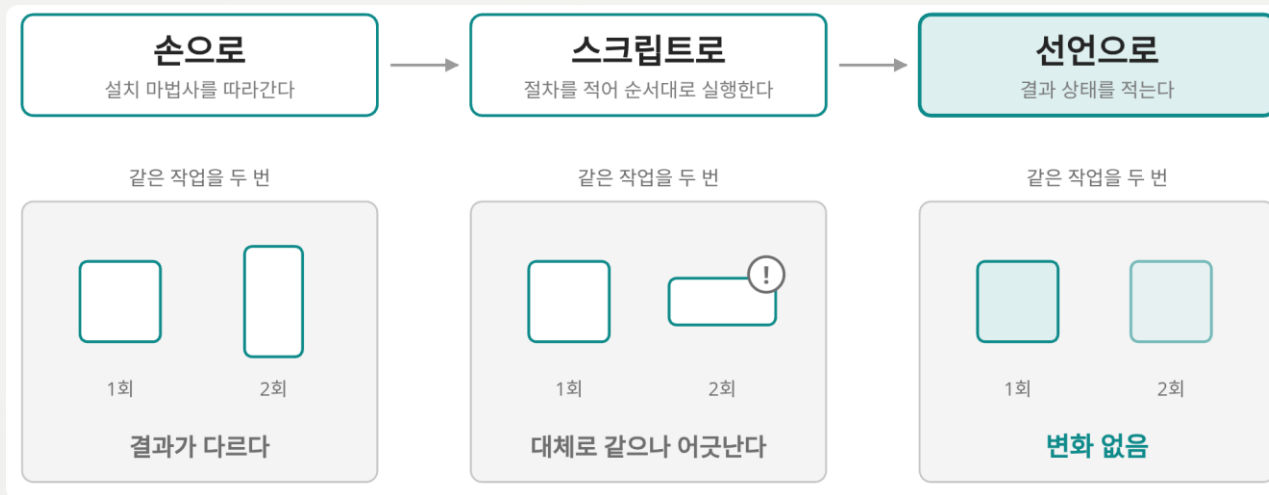
이 과목의 축은 제품이 아니라 자리다 (1주차 「유형 판단력」의 도구 버전)

새 도구를 만나면 묻는다 — 네 자리 중 어디에 들어가는가

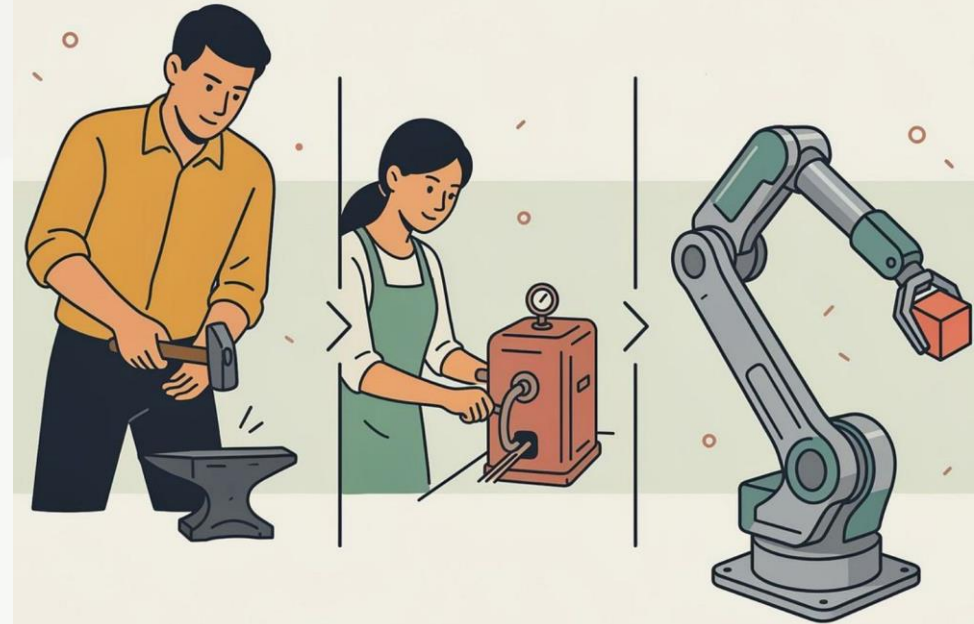
어느 자리에도 안 들어가면 그것은 도구가 아니라 아직 문제다

손에서 스크립트로, 스크립트에서 선언으로

시스템을 세우는 방식은 세 단계를 거쳐 옮겨 왔다.

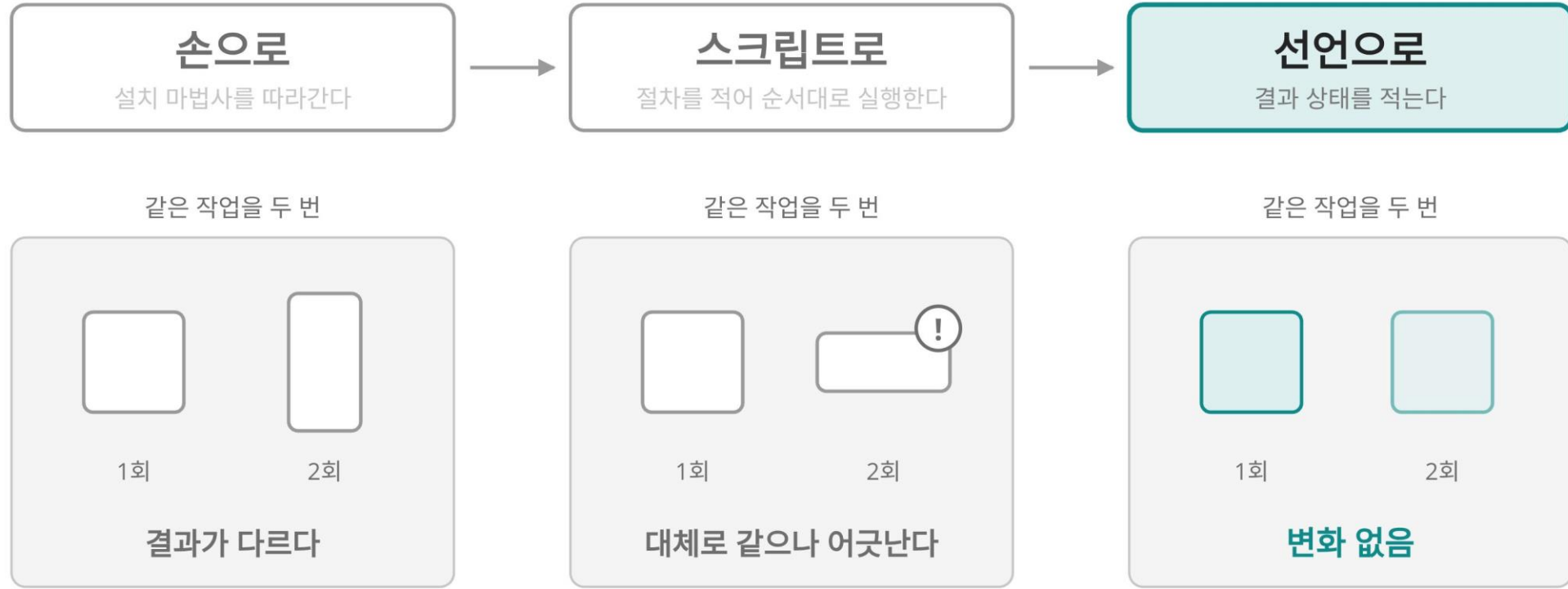


손으로 설치 마법사를 따라간다. 기록이 사람 머릿속에만 남는다	스크립트로 절차를 적는다. 순서대로 실행하면 대체로 같아진다	선언으로 결과 상태를 적는다. 어떻게 도달할지는 도구가 정한다
---	---	---



절차와 상태는 다르다

절차는 두 번 돌리면 달라지고, 상태는 두 번 돌려도 같다.



다시 돌려도 안전해야 자동으로 돌릴 수 있다

이 성질을 멍등이라 한다 — 자동화의 전제다

스크립트를 두 번 돌리면

디렉토리가 두 개 생기거나 실패한다

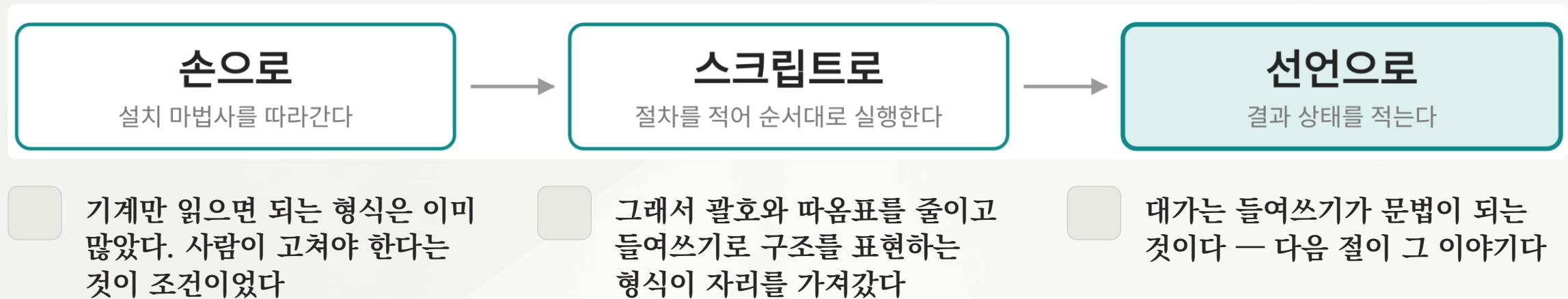
선언을 두 번 적용하면

이미 그 상태이므로 아무 일도 일어나지 않는다

이 성질 하나가 자동화의 전제다. 다시 돌려도 안전해야 자동으로 돌릴 수 있다

선언형이 요구한 대가

상태를 적으려면 사람이 읽고 고칠 수 있는 형식이어야 했다.



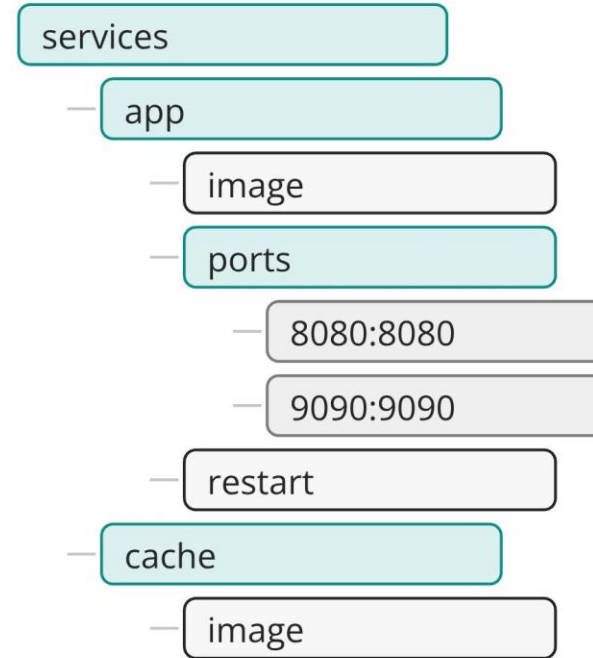
자료형은 셋뿐이다

아무리 긴 구성 파일도 스칼라·시퀀스·매핑 셋의 조합이다.

```

services:
  app:
    image: sample/app:1.4
    ports:
      - "8080:8080"
      - "9090:9090"
    restart: always
  cache:
    image: sample/cache:2.0
  
```

- ☒ 매핑 — 이름과 값의 짝
- ☐ 시퀀스 — 순서 있는 목록
- ☐ 스칼라 — 값 하나



색이 좌우로 이어진다 — 줄 하나가 구조의 한 칸이다

스칼라

값 하나. 문자열·숫자·참거짓

시퀀스

순서 있는 목록. 앞에 하이픈

매핑

이름과 값의 짝. 콜론으로 잇는다

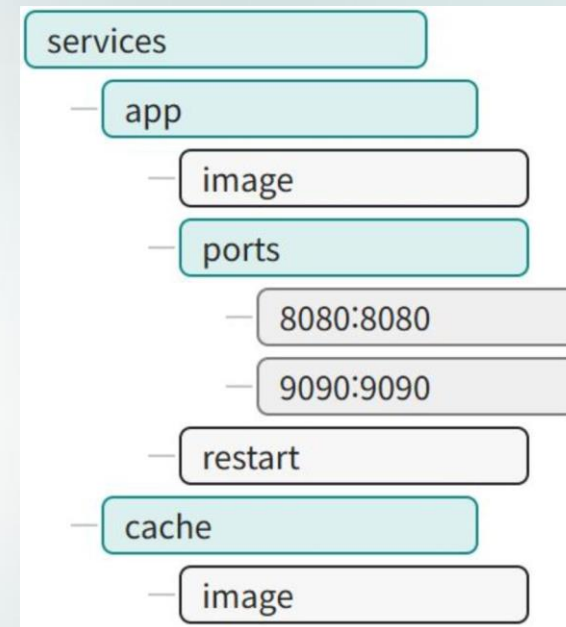
들여쓰기가 문법이다

다른 언어에서 보기 좋으라고 하는 들여쓰기가, 여기서는 구조 그 자체다.

같은 깊이면 형제, 더 깊으면 자식. 줄을 옮기면 의미가 바뀐다

탭은 금지다 — 폭이 환경마다 달라 같은 파일이 다르게 읽힌다

눈으로는 같아 보이는데 파싱이 깨지는 것이 이 형식의 대표 사고다



함정 셋

문법을 알아도 이 셋은 밟는다. 밟아 본 뒤에야 안 밟는다.

services:	매핑
app:	매핑
image: sample/app:1.4	스칼라
ports:	매핑
- "8080:8080"	시퀀스
- "9090:9090"	시퀀스
restart: always	스칼라
cache:	매핑
image: sample/cache:2.0	스칼라

- ☒ 매핑 — 이름과 값의 짝
직접 P1 에서 이 파일을 직접 쓴다 — 1주차에 명령어로 띄운 것을 파일로 옮긴다
- ☐ 시퀀스 — 순서 있는 목록

따옴표 없는 참거짓
no-off-yes 가 문자열이 아니라 참거짓으로 읽힌다

인용 없는 숫자형 문자열
앞자리 0 이 사라지고, 콜론이 든 값이 시간으로 읽힌다

후행 공백과 보이지 않는 문자
값 끝에 공백이 붙어도 화면에 표시되지 않는다

중복을 줄이는 장치, 그리고 그 대가

같은 블록을 여러 곳에 쓰는 장치가 있지만, 편해지는 만큼 추적이 어려워진다.

앵커와 별칭

한 블록에 이름을 붙이고 다른 곳에서 불러 쓴다

여러 문서를 한 파일에

구분선으로 나눈다

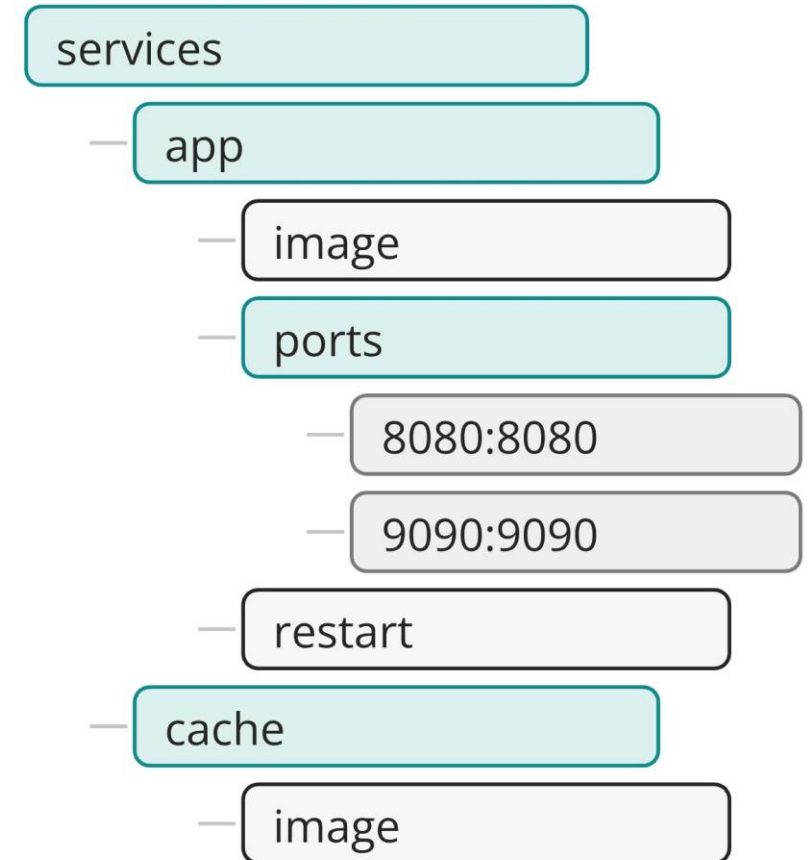
대가

한 곳을 고치면 여러 곳이 함께 바뀐다.
실습에서 쓸 때 다시 본다

실물 한 조각을 읽어 본다

컨테이너 구성 파일은 방금 배운 셋으로 전부 갈린다.

```
services:
  app:
    image: sample/app:1.4
    ports:
      - "8080:8080"
      - "9090:9090"
    restart: always
  cache:
    image: sample/cache:2.0
```



서비스 이름은 매핑의 키, 포트 목록은 시퀀스, 이미지 이름은 스칼라

어느 줄이 어느 자료형인지 색으로 갈라 보면 구조가 먼저 보인다

실습 P1 에서 이 파일을 직접 쓴다 — 1주차에 명령어로 띄운 것을 파일로 옮긴다

선언은 기계에게, 다음은 사람에게

1교시가 확정한 것은 기계가 읽을 형식까지다.

- 구성 파일은 무엇을 원하는지를 적는다. 왜 그렇게 정했는지는 안 적힌다
- 그 「왜」는 파일 어디에도 없고, 남기지 않으면 사라진다
- 2교시는 그 자리를 다룬다

2교시 — 기록

今日もまた、静かな朝。

窓の外は雨の音。

コーヒーを淹れた。

本を開いたが、

読む気になてず。

ただ、この紙に

書き留めるだけ。

—— 午後三時

사라지는 것은 코드가 아니다

코드는 무엇을 하는지 말하지만, 왜 그렇게 했는지는 말하지 않는다.

- ☐ 6개월 뒤 코드를 읽으면 동작은 알 수 있고 판단은 알 수 없다
- ☐ 특히 사라지는 것은 하지 않기로 한 것 — 검토했다가 뻔 선택지
- ☐ 뻔 이유가 없으면 다음 사람이 그것을 다시 넣는다

같은 결정을 두 번 논의한다

기록이 없을 때 드는 비용은 시간이 아니라 판단의 반복이다.

두 번째 논의는 첫 번째의 조건을
모른 채 시작한다

조건이 그대로면 같은 결론에 다시
도달하고, 조건이 바뀌었으면
그것을 알 방법이 없다

조사한 실운영 시스템 4건 중 결정
기록 체계를 갖춘 것은 소수다
(전수는 문안 확정 후)

문서는 코드와 같은 곳에 산다

코드와 문서가 떨어지는 순간 문서는 낡기 시작한다.

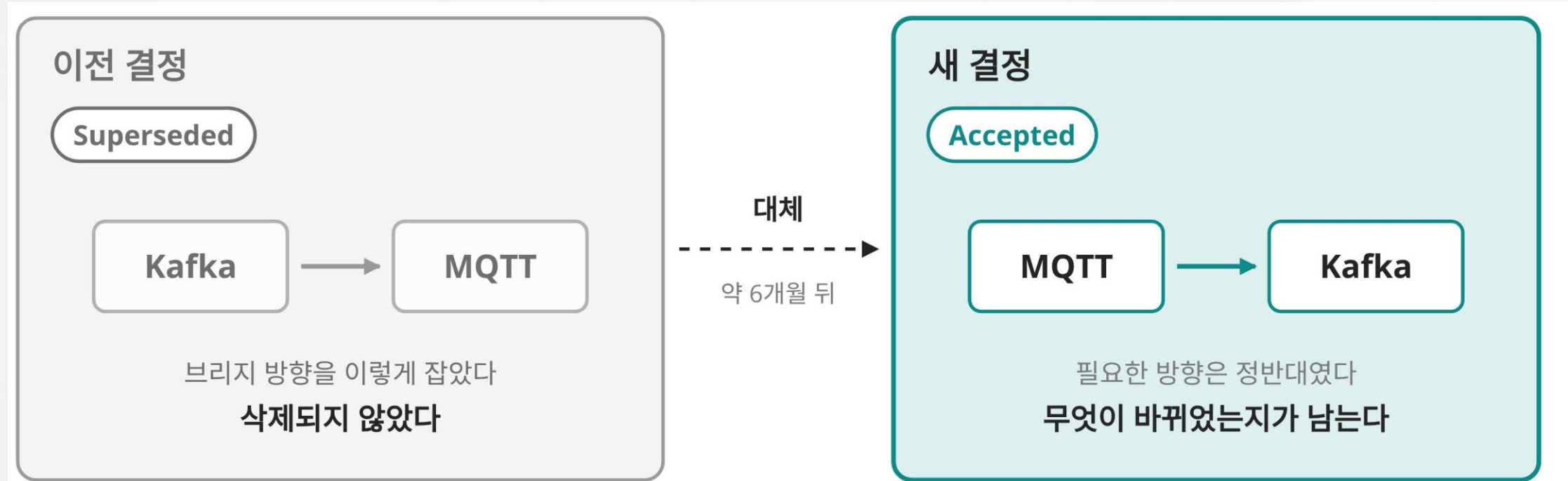
같은 저장소, 같은 커밋 — 코드를
고치는 사람이 문서도 같이 고친다

별도 위키·공유 드라이브에 두면
고칠 사람과 고칠 시점이 사라진다

텍스트로 두는 이유도 같다 — diff
가 되어야 리뷰에 걸린다

뒤집힌 결정을 지우지 않는다

아키텍처 결정은 반복된다. 문제는 반복 여부가 아니라 반복이 기록으로 남는가다.



방향이 뒤집힌 원인은 판단 오류가 아니라 외부 조건의 확정이다 — 결정 시점에 확정되지 않은 조건이 있었다는 사실 자체가 기록돼 있다

지운 결정은 3년 뒤 다시 검토된다. 남은 결정은 다시 검토되지 않는다

재가복지 시스템 — 결정 기록 8건이 남아 있는 사례

초기 결정은 한 방향이었는데, 약 6개월 뒤 필요한 방향이 정반대임이 드러났다

옛 결정 문서는 삭제되지 않고 Superseded(대체됨)로 남았다

방향이 뒤집힌 원인은 판단 오류가 아니라 외부 조건의 확정이다. 결정 시점에 확정되지 않은 조건이 있었다는 사실 자체가 기록돼 있다.

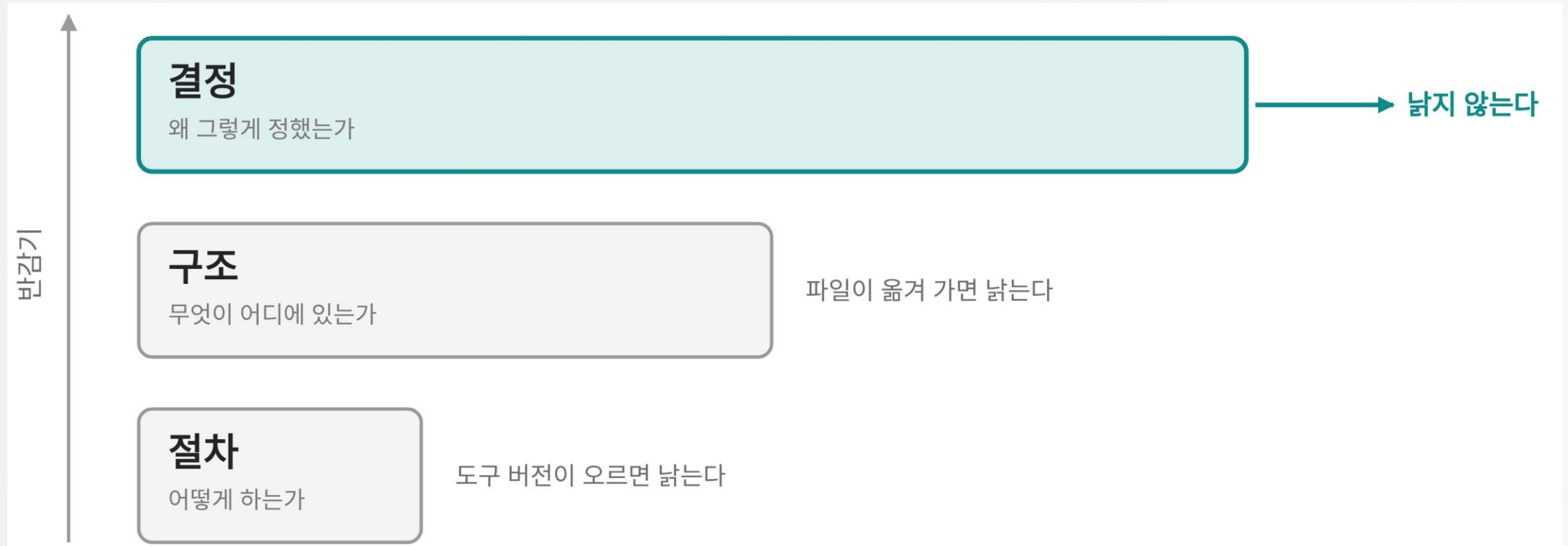
문법은 짧다. 어려운 것은 다른 데 있다

마크업 문법은 한 시간이면 끝난다. 무엇을 어디에 쓰는가가 남는다.

- ☐ 제목·목록·표·코드 블록 — 필요한 것은 이 정도다
- ☐ 문법을 다 알아도 빈 파일 앞에서 막힌다
- ☐ 막히는 이유는 문법이 아니라 체계가 없기 때문이다

문서는 세 층이다

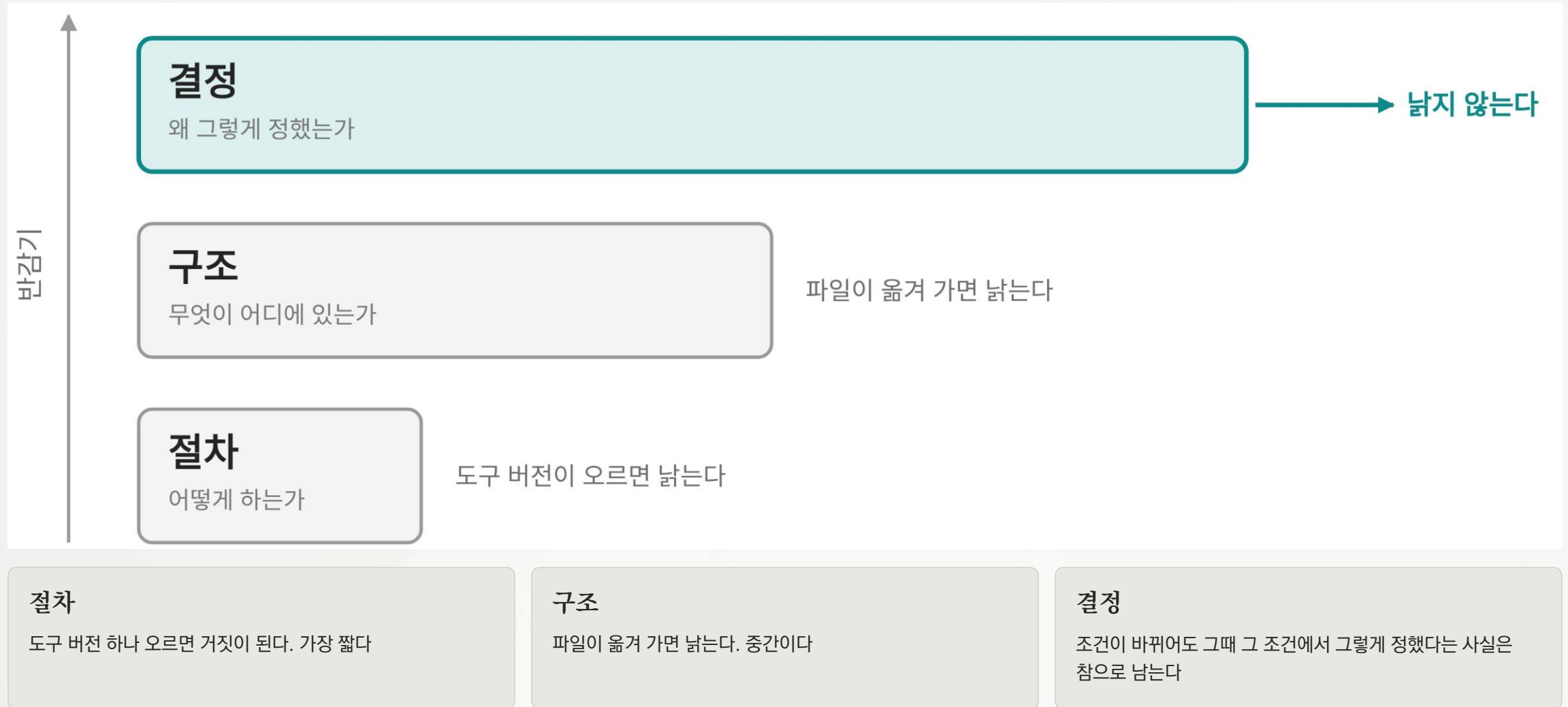
결정·구조·절차를 섞으면 셋 다 남는다.



결정은 조건이 바뀌어도 '그때 그 조건에서 그렇게 정했다' 가 참으로 남는다.

층마다 반감기가 다르다

절차는 가장 빨리 낡고, 결정은 낡지 않는다.



결정 기록의 네 칸

배경·결정·근거·영향. 네 칸이 각각 다른 실패를 막는다.

배경 없으면

결정이 조건과 분리된다 — 조건이 바뀌어도
결정만 남는다

근거 없으면

반박할 수 없다 — 논의가 취향 싸움이 된다

영향 없으면

결정이 어디까지 번지는지 모른다 — 고칠 때
무엇이 달려오는지 모른다

그림도 텍스트로 둔다

그림 파일은 diff 가 안 되고, diff 가 안 되면 고쳐지지 않는다.

- 1 텍스트로 쓴 다이어그램은 코드처럼 리뷰에 걸린다
- 2 그림 파일은 누가 언제 무엇을 바꿨는지 남지 않는다
- 3 정교한 도판이 필요할 때만 파일로 만들고, 나머지는 텍스트로

전부 쓰면 아무것도 안 쓴 것과 같다

고쳐지지 않는 문서는 없느니만 못하다. 거짓이 되기 때문이다.

- ☐ 문서가 많을수록 고치는 비용이 늘고, 고쳐지 않은 문서는 틀린 정보가 된다
- ☐ 없는 문서는 사람을 찾아가게 하지만, 틀린 문서는 그대로 믿게 만든다
- ☐ 그래서 분량을 제한하는 것이 성실함의 반대가 아니다



남긴다 / 버린다

기준은 하나다. 이것을 모르면 다음 사람이 막히는가.

남긴다	버린다
되돌릴 수 없는 결정	코드를 보면 아는 것
두 번 물어본 것	도구가 알려주는 것
다음 사람이 알 수 없는 것	한 번 쓰고 안 볼 것

기준은 고정이지 아니다 — 팀 크기와 프로젝트 수명이 바뀌면 목록도 뒤집힌다

고쳐지지 않는 문서는 없느니만 못하다. 거짓이 되기 때문이다

남긴다

- 되돌릴 수 없는 결정
- 두 번 물어본 것
- 다음 사람이 알 수 없는 것

버린다

- 코드를 보면 아는 것
- 도구가 알려주는 것
- 한 번 쓰고 안 볼 것

기준은 고정이지 아니다 — 팀 크기와 프로젝트 수명이 바뀌면 목록도 바뀐다

정본을 가리키는 인덱스 하나

파일이 있다는 것이 그것이 정본이라는 뜻은 아니다.

- 1 문서가 쌓이면 어느 것이 유효한지가 문법보다 어려운 문제가 된다
- 2 인덱스 한 장이 정본·폐기·대기를 가린다. 그 인덱스는 반드시 고쳐야 한다
- 3 이것이 절제 판단의 도구 버전이다 — 1주차 「언제 필요 없는가」와 같은 축

여기까지가 내 손 안

1·2교시가 다룬 것은 내 컴퓨터 안의 파일이다.

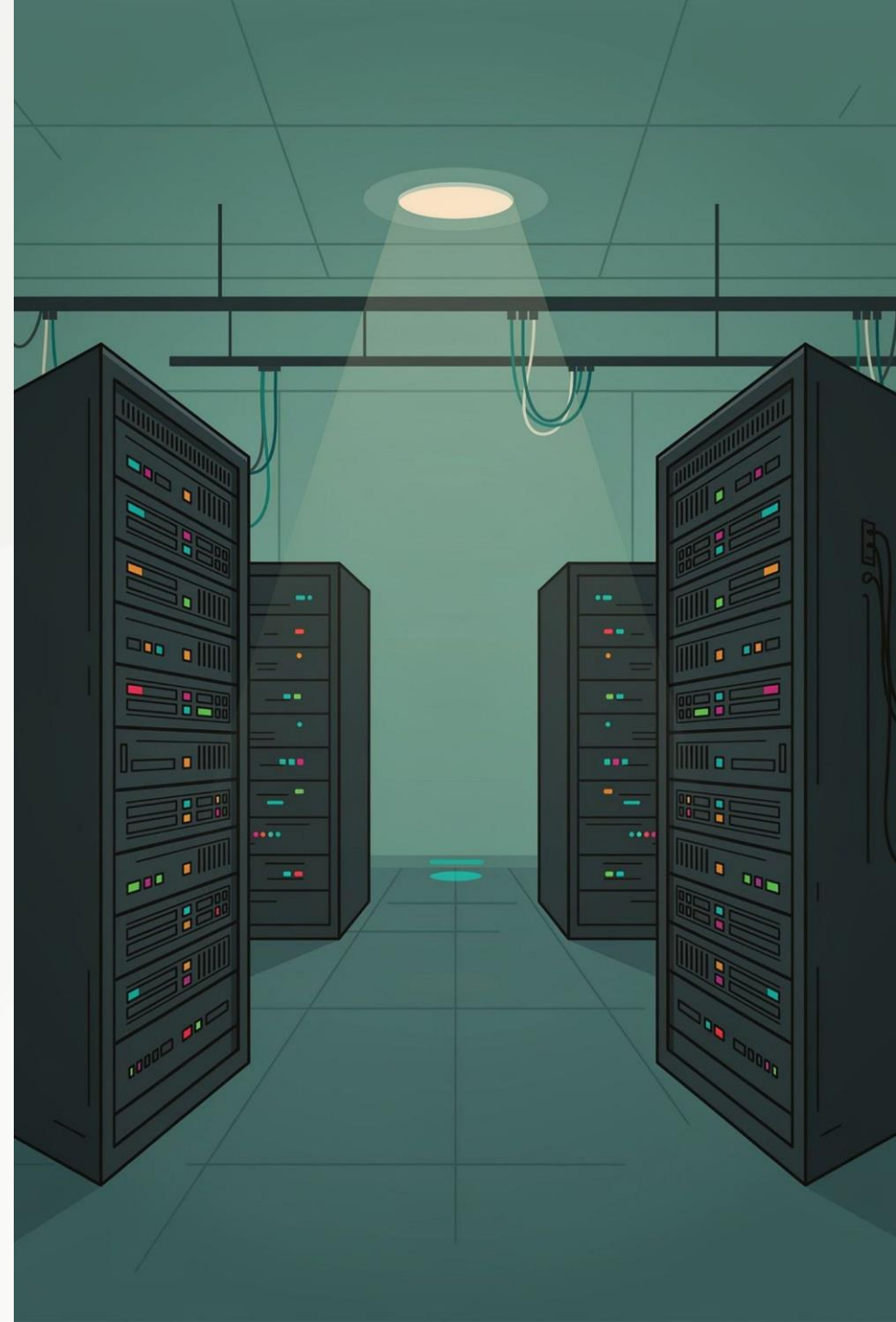
- ☐ 구성 파일도 문서도 내 손 안에 있다. 열고 고치는 데 아무 장벽이 없다
- ☐ 그런데 실제 시스템은 다른 곳에 있다
- ☐ 3교시는 저쪽에 손을 대는 방법이다

3교시 — 접속과 관찰

시스템은 내 PC 에 없다

12주 내내 우리가 다루는 것은 다른 곳에서 도는 것들이다.

- ☐ 서버실·클라우드·다른 사람의 컨테이너 — 물리적으로 손이 닿지 않는다
- ☐ 화면도 키보드도 없는 기계와 대화해야 한다
- ☐ 원격이 예외가 아니라 기본값이라는 전제에서 도구가 만들어졌다



들어간다 / 당겨온다 / 불러낸다

원격을 다루는 방법은 셋이고, 3교시 구성이 그 셋을 따른다.

들어간다

원격 셀로 저쪽 안에 선다

당겨온다

저쪽 포트를 이쪽으로 끌어와 내 도구로 본다

불러낸다

API 로 요청을 보내고 답을 받는다

비밀번호가 왜 한계인가

사람이 기억하는 것은 자동화가 쓸 수 없다.

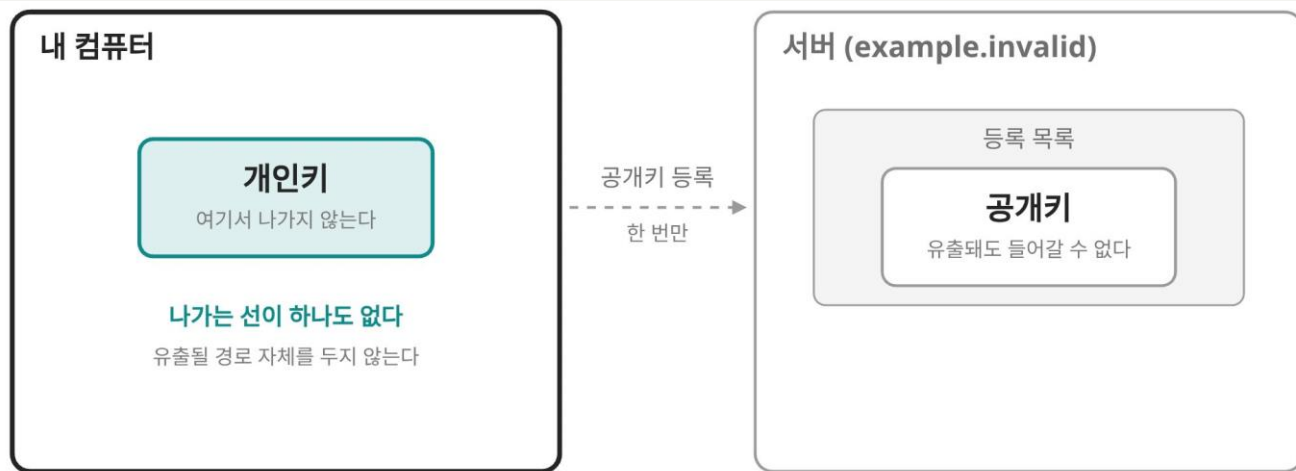
스크립트가 밤에 도는데 비밀번호를 물으면 거기서 멈춘다

스크립트에 적어 두면 그 파일이 곧 열쇠가 된다

호스트가 늘수록 관리 대상이 사람 수 $\times$ 호스트 수로 늘다

키 쌍 — 비대칭이 전부다

개인키는 내 손을 떠나지 않고, 공개키만 서버로 간다.



개인키 파일 권한이 느슨하면 서버가 거부한다 — 클라이언트 쪽 문제인데 서버 오류로 보인다

화면의 호스트명·키는 전부 가상값이다

개인키

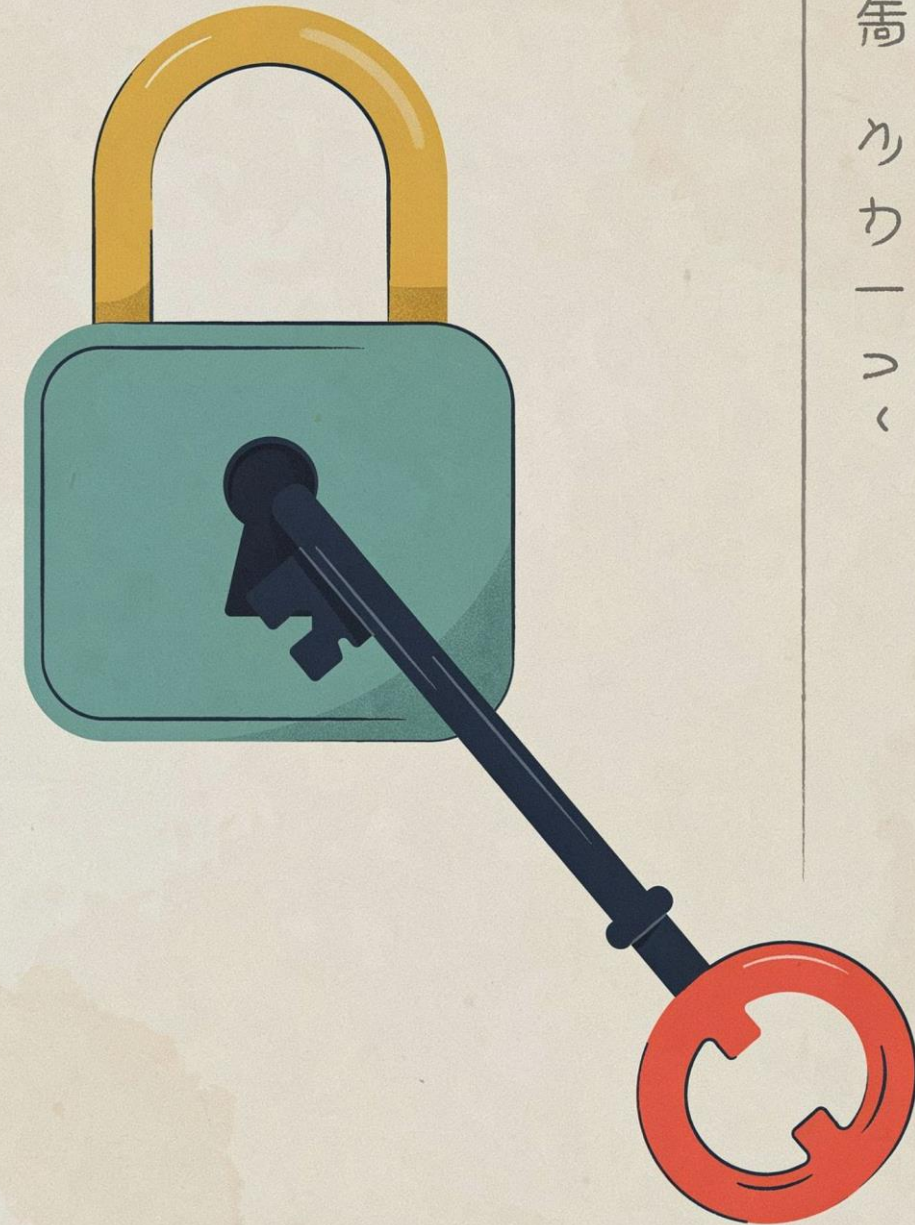
내 컴퓨터에만 있다.
어디에도 보내지 않는다

공개키

서버에 등록한다. 유출돼도
그것만으로는 들어갈 수
없다

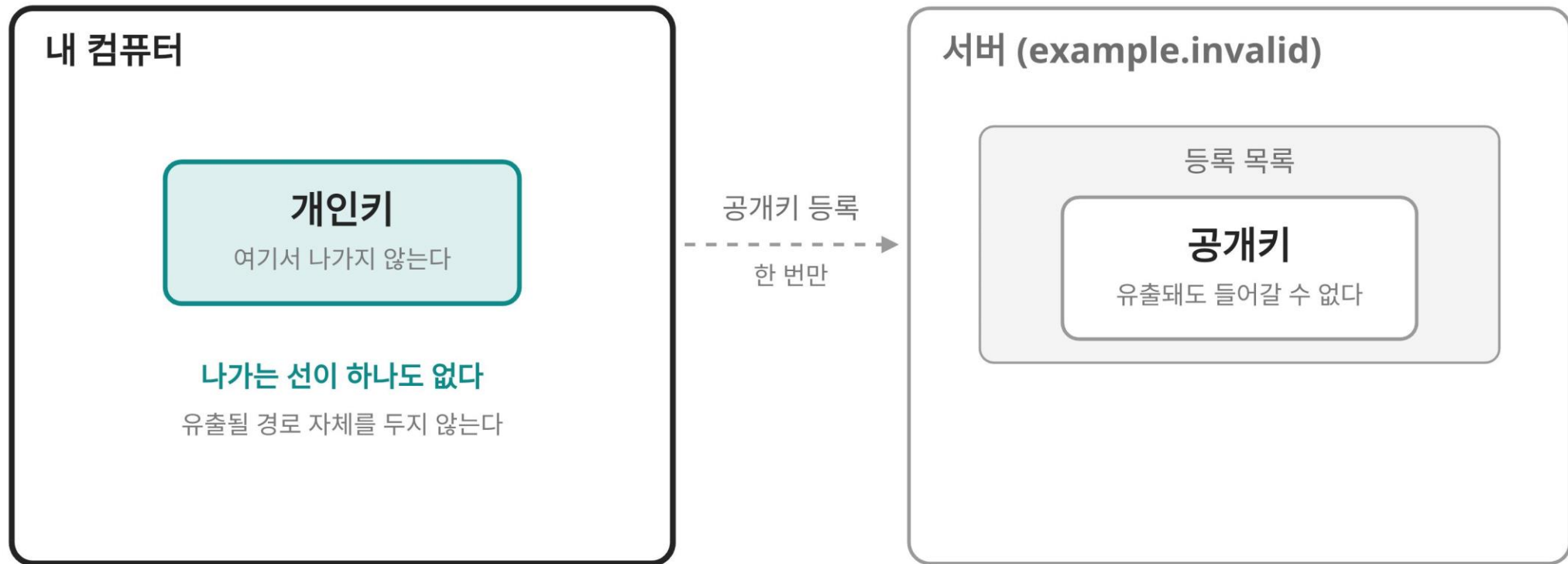
비대칭

비밀을 주고받지 않고
신원을 증명하는 것 — 이
비대칭이 전부다



서버는 무엇을 확인하는가

서버는 등록된 공개키로 낸 문제를 개인키로만 풀 수 있는지 본다.



① 목록에서 내 공개키를 찾는다

② 그 키로만 풀리는 문제를 낸다

③ 답이 맞으면 통과 — 비밀은 오가지 않았다

1 서버가 등록 목록에서 내 공개키를 찾는다

2 그 공개키로만 확인 가능한 증명을 요구하고, 개인키를 가진 쪽만 답할 수 있다

3 개인키 파일 권한이 느슨하면 서버가 거부한다 — 클라이언트 쪽 문제인데 서버 오류로 보인다

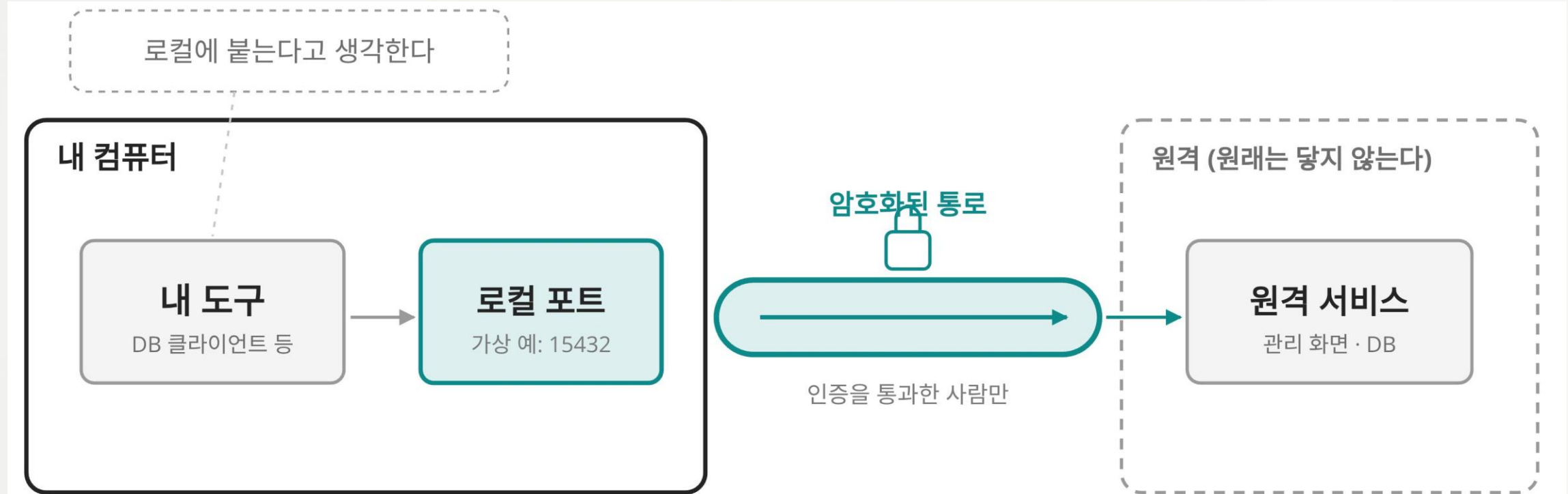
호스트 별명 — 접속 정보를 설정으로 분리한다

주소와 사용자와 키 경로를 명령에 매번 넣지 않고, 설정 파일에 이름 하나로 묶는다.

- ☐ 별명 하나에 주소·사용자·포트·키 경로를 묶는다
- ☐ 접속 정보가 명령이 아니라 설정에 사는 것 — 1교시 선언 논지와 같은 축이다
- ☐ 12주 내내 쓴다. 실습 P3 의 통과선이 여기다

저쪽 포트를 이쪽으로 당긴다

원격 서비스를 내 컴퓨터의 포트처럼 쓰는 통로를 낸다.



열어 두지 않는다 — 필요할 때만 통로를 낸다. 8주차·11주차에서 실제로 쓴다

터미널을 닫으면 끊긴다. 끊기는 것이 결함이 아니라 성질이다 — 터널은 세션이지 구성이 아니다

- ☐ 내 로컬 포트에 들어온 연결이 암호화된 통로를 지나 원격 서비스에 닿는다
- ☐ 내 도구는 로컬에 붙는다고 생각하고 동작한다 — 도구를 고칠 필요가 없다
- ☐ 방향을 뒤집는 형태와 프록시처럼 쓰는 형태도 있다. 이름만 건다

인터넷에 열지 않고 보는 방법

관리 화면을 외부에 공개하는 대신, 필요할 때만 통로를 낸다.

컨테이너 관리 UI·DB·모니터링 화면은 열어 두면 전부 공격면이다

포워딩은 인증을 통과한 사람만 그 순간 통로를 갖는다

8주차·11주차에서 실제로 쓴다 — 저장소 관리 화면과 관제 대시보드

터널은 세션이지 구성이 아니다

상시로 두어야 한다면 그것은 터널이 답이 아니라는 신호다.

- ☐ 터미널을 닫으면 끊긴다. 끊기는 것이 결함이 아니라 성질이다
- ☐ 상시 접근이 필요하다면 다른 유형이 답이다 — 그 도구는 11주차 소관
- ☐ 임시로 쓰는 것과 구성으로 두는 것의 경계를 아는 것이 판단이다

GUI 가 대신해 주는 것

클라이언트 도구는 탐색을 빠르게 한다.

GUI 가 대신한다	대신하지 않는다
스키마 탐색	반복 — 매일 돌릴 수 없다
결과를 표로	기록 — 무엇을 언제 조회했는지
접속 정보 보관	자동화 — 이어 붙일 수 없다

한 번 볼 것은 GUI, 두 번 이상 볼 것은 스크립트

스키마 탐색

어떤 테이블이 있고 어떻게 이어지는지 클릭으로 본다

결과를 표로

정렬·필터를 즉석에서

접속 정보 보관

여러 대상을 한 창에서

대신해 주지 않는 것

반복·기록·자동화는 GUI 가 못 한다. 그래서 CLI 가 사라지지 않는다.

GUI 가 대신한다	대신하지 않는다
스키마 탐색	반복 — 매일 돌릴 수 없다
결과를 표로	기록 — 무엇을 언제 조회했는지
접속 정보 보관	자동화 — 이어 붙일 수 없다

한 번 볼 것은 GUI, 두 번 이상 볼 것은 스크립트

반복

같은 조회를 매일 돌릴 수 없다

기록

무엇을 언제 조회했는지 남지 않는다

자동화

다른 작업에 이어 붙일 수 없다

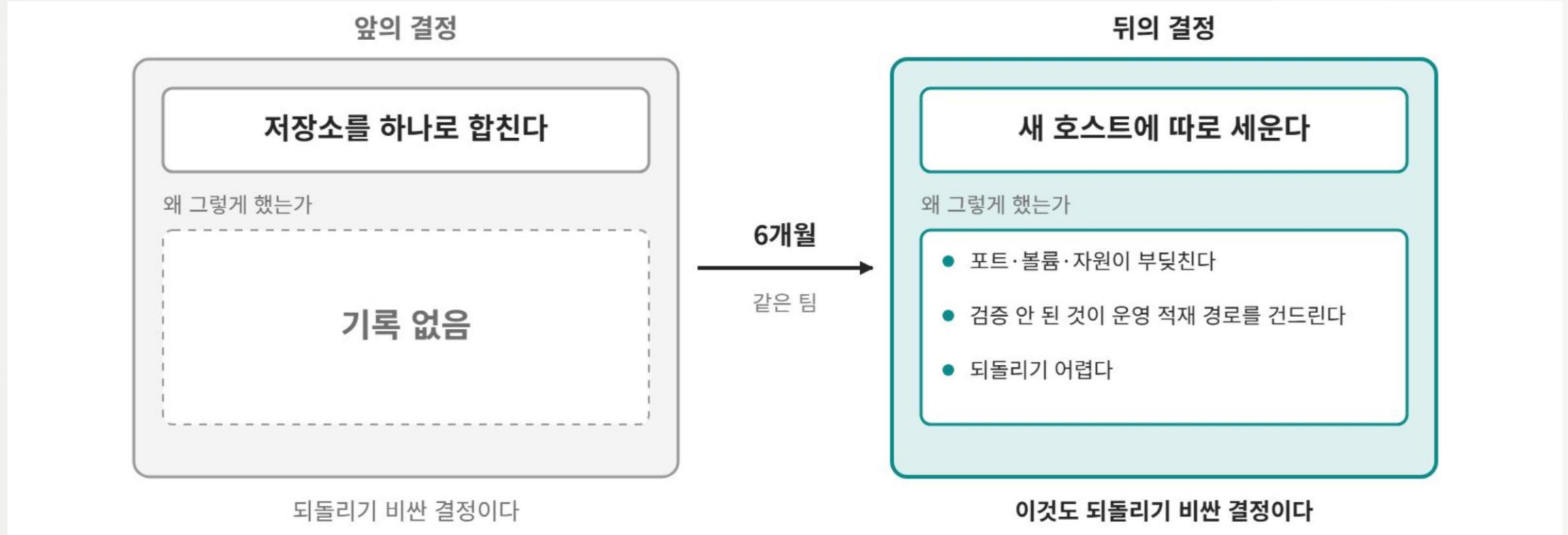
한 번 볼 것은 GUI, 두 번 이상은 스크립트

판단 기준은 이 한 줄이다.

- ☐ 탐색은 GUI 로, 확정된 질의는 파일로 옮긴다
- ☐ GUI 가 만들어 준 질의를 그대로 쓰지 않는다 — 사람이 읽도록 만들어지지 않았다
- ☐ API 검증 도구도 같다 — 시험은 도구로, 확정된 호출은 스크립트로

같은 시스템, 6개월 간격의 두 결정

두 결정 사이에 바뀐 것은 팀도 도구도 아니다. 결정을 적는 자리가 생긴 것뿐이다.



앞의 결정 — 흩어져 있던 저장소를 하나로 합쳤다. 왜 합쳤는지는 커밋 어디에도 없다

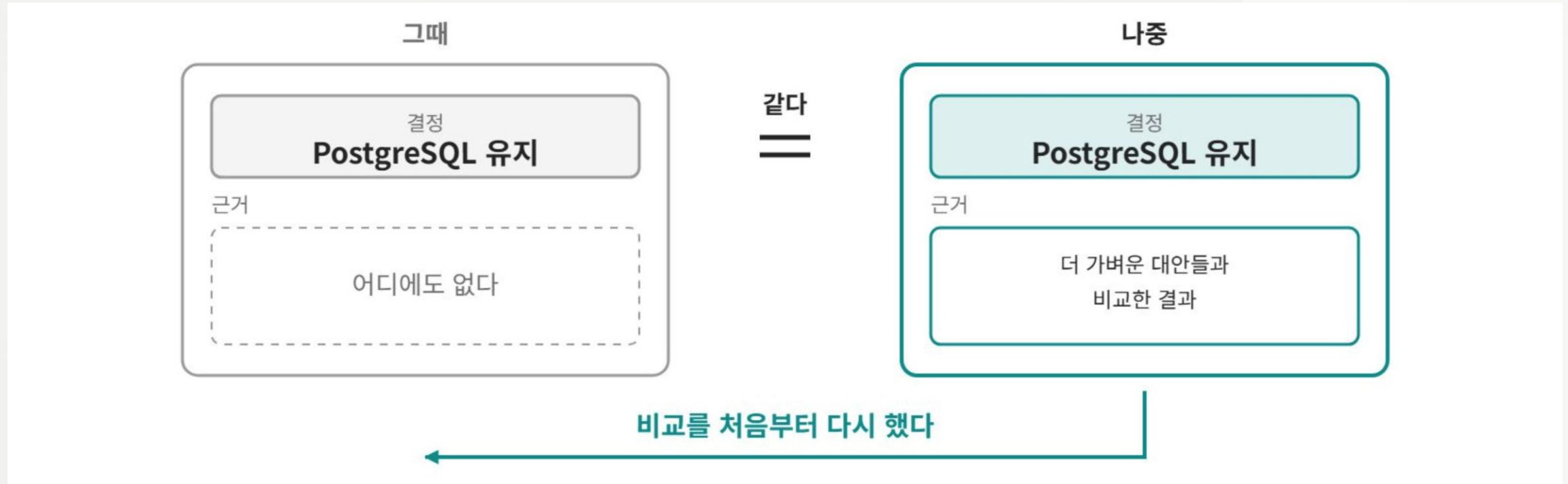
뒤의 결정 — 새 스택을 운영 중인 호스트에 없지 않고 새 호스트에 따로 세웠다. 이유가 셋, 문장으로 남았다 —
포트·볼륨·자원 충돌 / 검증 안 된 것이 운영 적재 경로를 건드림 / 되돌리기 어려움

앞의 결정도 되돌리기 비싼 결정이었다. 다른 것은 적어 두었느냐뿐이다

앞의 결정 시점은 초기 구축기다. 구성요소가 적고 만지는 사람이 적은 동안에는 기록의 이득이 비용을 넘지 않는다. 규모가 자라면서 기준이 바뀐 것이지 앞의 결정이 잘못된 것이 아니다.

결론은 그대로였는데, 다시 해야 했다

잃은 것은 결정이 아니라 근거다. 근거가 없으면 결론이 맞아도 다시 증명해야 한다.



그 시스템은 저장소로 PostgreSQL 하나를 쓴다. 설계서에도 그렇게 적혀 있었다

그런데 왜 그것이어야 하는지가 어디에도 없었다 — 더 가벼운 대안들과 비교한 기록이 없다

그래서 결정 기록 첫 문서가 그 비교를 새로 해서 채웠다. 결론은 바뀌지 않았다 — PostgreSQL 그대로다

근거를 뒤늦게 채운 것은 결함의 수습이 아니라 기록 체계를 세우는 과정의 일부다. 되찾을 수 있었던 것은 결정을 내린 사람들이 아직 그 자리에 있었기 때문이다.

어디에 적었느냐가 무엇이 남는지를 정한다

기록은 있거나 없거나가 아니다. 네 단계가 있고, 어느 단계에도 남지 않는 것이 하나 있다.



- 아무 데도 없음 — 터널 화재감지 시스템은 화재를 판정하는 추론 모델을 통째로 교체했는데 커밋도 이슈도 없다
- 커밋 제목만 — 전광판 감시 시스템의 「프로토콜 자동 감지 엔진 도입」. 왜 자동 감지였는지는 없다
- 커밋 본문·주석 — 위치측위 시스템은 동기 호출을 바꾸면서 원인을 숫자까지 커밋에 적었다. 문서는 없어도 이유는 남았다
- 네 단계 어디에도 없는 것이 하나 있다 — 검토했다가 버린 대안

네 단계는 우열이 아니라 비용이 다른 선택지다. 결정이 자주 뒤집히지 않고 사람이 바뀌지 않는 구간에서는 왼쪽 두 칸으로 충분하다.

오늘 배운 넷을 손으로 한 번씩

실습 90분에 선언·기록·접속·관찰을 각각 한 번씩 밟는다.

선언

1주차에 명령어로 띄운 것을 구성 파일로

기록

문서 3층과 결정 1건

접속

학과 서버에 키로

관찰

DB 클라이언트로 들여다보기

실습 환경은 이 PC 그대로다. 서버는 접속 대상일 뿐 실습 자리가 아니다